LEADERS UNIVERSITY / LICENCE BI

Diriger Codex et travailler à deux

Prompts, modèle et effort, revue, Git au quotidien, dépannage · le livret à ouvrir à chaque demande et à chaque pull request

18	8	0
PROMPTS PRÊTS À ADAPTER	HABITUDES DE TRAVAIL	LIGNE DE CODE À LA MAIN

SOLEN DIGITAL · CLIENT FICTIF MAISON ZITOUNA

Demande → diff → contrôle → revue à deux → fusion

03 / DIRIGER CODEX ET GIT

Youssef Douma · Leaders University

Édition pédagogique · 28 octobre 2026 · données fictives

03 / DIRIGER CODEX · SOMMAIRE

Sommaire : à ouvrir à chaque demande et à chaque PR

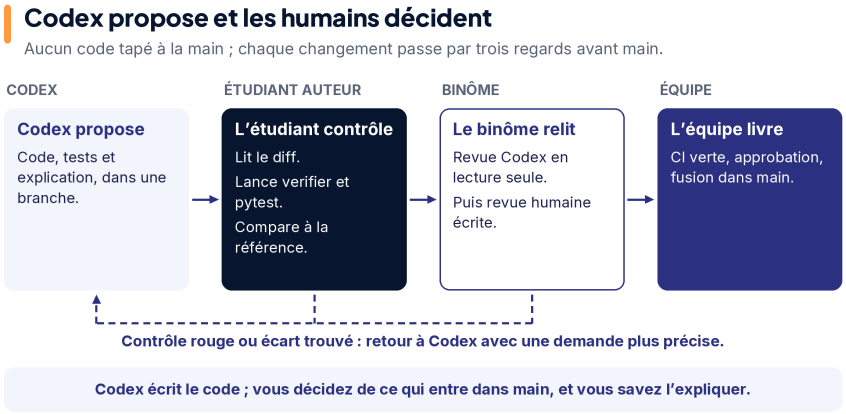
Partie	Ce que vous y trouvez	Page
1. Codex propose, vous décidez	Qui fait quoi ; ce que Codex ne fait jamais.	3
2. Modèle, effort, vitesse, tokens, contexte, quota	Six mots ; quel effort pour quelle tâche.	4
3. Les huit habitudes	De la demande précise à la revue.	6
4. Les prompts prêts à adapter	Les 18 fiches, par moment (ci-dessous).	9
5. Lire un diff et un fichier Python	Fonction, module, exception, classe ; où regarder.	27
6. La revue à deux	$revoir-bi, puis votre revue écrite sur GitHub.	29
7. Git au quotidien	Le cycle en liste de contrôle ; le dépannage.	30
8. Codex indisponible ou quota atteint	Ce qui continue sans IA ; le point de reprise.	33
9. Erreurs Python et contrôles rouges	Les messages fréquents et quoi demander.	34
À ce stade, je sais…	Auto-contrôle de fin de livret.	36

Moment	Les 18 prompts (page)
Cadrer et planifier (S1 → S5)	01 Cadrer les indicateurs (9) · 02 Plan court (10)
Construire (S2 → S5)	03 Réaliser une petite étape (11) · 04 Lire CSV, Excel, JSON (12) · 05 Nettoyer et rejeter (13) · 06 SQL paramétré (14) · 07 Charger l'étoile (15) · 08 Incrément, SCD2, rejeu (16) · 09 Tests et CI (17)
Revue et PR (chaque PR)	11 Revue en lecture seule (18) · 12 Corriger après revue (19) · 13 Préparer la PR (20)
Quand ça bloque	10 Comprendre une erreur (21) · 15 Résoudre un conflit (22)
Expliquer et reprendre	14 Ligne source → graphique (23) · 16 Note d'état et reprise (24)
Aller plus loin (S4, mission)	17 Skill local (25) · 18 Variation métier (26)

03 / DIRIGER CODEX · LES RÔLES

1. Codex propose, vous décidez

Codex lit votre dépôt, propose du code, lance une commande et montre ce qu'il a changé. Il se trompe parfois avec assurance : le cours rend ses erreurs **visibles** (diff lu, contrôle relancé, revue par une autre personne).



Codex propose dans une branche ; l'auteur contrôle ; le binôme relit (Codex en lecture seule, puis revue humaine) ; l'équipe livre après CI verte et approbation.

Qui	Son rôle	Ce qu'il ne fait jamais
Codex	Écrit Python, SQL, DAX, tests, YAML dans `solen/`, `tests/`, `powerbi/`, `.github/` ; lance une commande, recopie la sortie, explique.	Modifier sources, `reference/`, `controles/` ; commiter, pousser, ouvrir ou fusionner une PR.
Vous, l'auteur	Rédigez la demande, prévoyez une valeur, relancez les contrôles, lisez le diff, décidez, faites Git.	Taper du code ; croire un « ça marche » sans sortie réelle.
Votre binôme	Récupère la branche, relance les contrôles, fait relire Codex en lecture seule, écrit sa revue GitHub.	Approuver sans rien reproduire.
Vérificateur et CI	Comparent vos sorties à la référence, chez vous puis sur une machine Linux neuve.	Lire votre code.

Trois règles qui ne changent jamais

1. Vous ne tapez jamais de code. 2. Une valeur n'est vraie que lorsque **vous** avez relancé la commande qui la montre : « 26 rejets » annoncé par Codex reste une promesse tant que vous n'avez pas lu la ligne du vérificateur. 3. Git et GitHub sont vos gestes.

Repères : <…> = à remplacer, chevrons compris. Terminal Ubuntu, dossier `~/solen-bi`, venv actif. Exemples : **Solen Commerce 360** (Maison Zitouna, boutiques TUN, NBE, SFX).

03 / DIRIGER CODEX · MODÈLE, EFFORT, VITESSE, TOKENS, CONTEXTE, QUOTA

2. Six mots différents

Les confondre mène à de mauvaises décisions. Seul ce qu'affiche un **compte Free** compte ici.

Mot	Ce que c'est	Où le voir · le bon geste
Modèle	Annoncé pour Free : **GPT-6 Luna**, selon le déploiement.	Sélecteur de modèle (Ctrl + Maj + M) · choisir Luna.
Effort	Combien il réfléchit : **Light, Medium, High, Extra High**.	Sous **Advanced** (« Luna High » possible) · commencer **High**.
Vitesse	« Standard » : une vitesse de service, pas un effort.	Libellé de l'offre Free · rien à régler.
Tokens	Morceaux de texte : demande, fichiers lus, réponse.	Dans l'usage consommé · peu de fichiers, une petite étape.
Contexte	Ce que la conversation contient ; trop plein, Codex oublie.	`/status` · note d'état (prompt 16), conversation neuve.
Quota	Allocation d'usage sur une période, variable.	`/status`, tableau d'usage · limite : section 8.



Tokens, contexte, effort et quota : quatre angles sur une même demande.

Ce qu'on ne vous promet pas

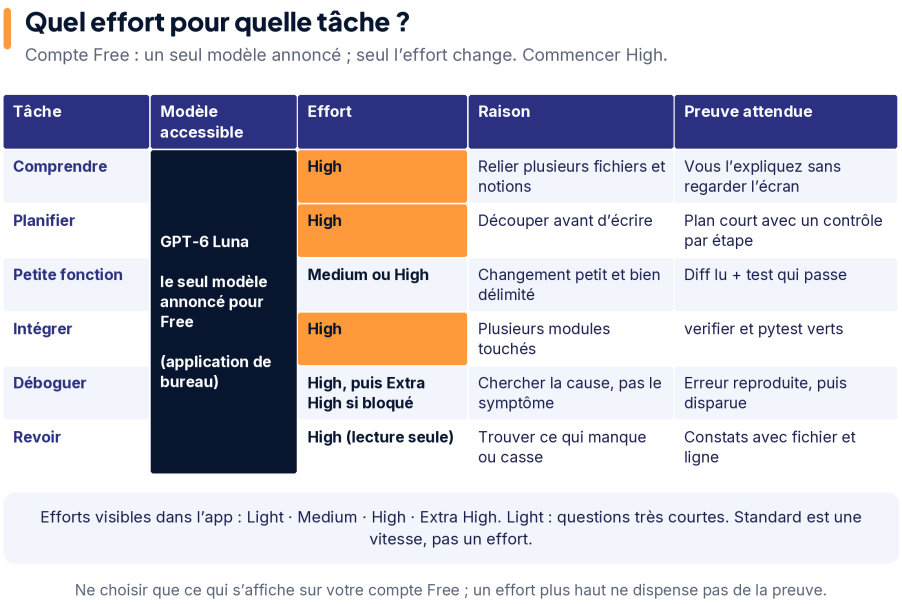
Ni nombre de tokens, ni « 4 h de Codex ». Monter l'effort ne vide pas un contexte surchargé, et une conversation neuve ne rend pas de quota. Lisez l'usage affiché ; notez-le au journal.

03 / DIRIGER CODEX · LA FICHE TÂCHE

Quel effort pour quelle tâche ?

La **fiche tâche** : tâche → modèle accessible → effort accessible → raison → preuve attendue. Sur Free, seul l'effort varie ; un effort plus haut ne dispense jamais de la preuve.

Tâche (exemple Solen)	Modèle	Effort	Raison	Preuve attendue
Comprendre le grain de `fact_vente`	Luna	High	Relier fichiers et notions	Vous l'expliquez sans l'écran
Planifier `outputs/rejets.csv`	Luna	High	Découper avant d'écrire	3 à 5 étapes, un contrôle chacune
Petite fonction : `18,500` → 18500	Luna	Medium ou High	Petit, bien délimité	Diff lu + test vert
Intégrer `python -m solen initial`	Luna	High	Plusieurs modules	`--etape initial` OK
Déboguer « attendu 1960, observé 1966 »	Luna	High → Extra High si bloqué	La cause, pas le symptôme	Erreur reproduite puis disparue
Revoir avec `$revoir-bi`	Luna	High	Trouver ce qui manque	Constats fichier:ligne
Question courte : « valid_to ? »	Luna	Light	Rien à relier	Vous reformulez



La même fiche en image : un seul modèle annoncé pour Free ; on commence High.

03 / DIRIGER CODEX · HABITUDES 1 À 3

3. Les huit habitudes

1 · Être précis

Un objectif observable (un fichier, un compte, un total), les fichiers concernés et ceux à ne pas toucher, la commande de contrôle et la valeur que **vous** prévoyez.

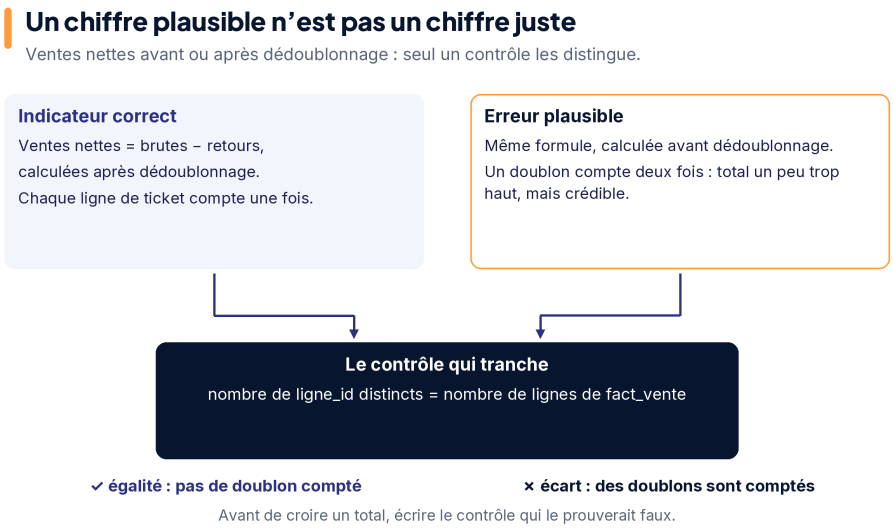
Demande floue	Demande précise (exemple Solen)
« Fais le nettoyage des ventes. »	« Dans `solen/`, rejette les lignes de `ventes.csv` à quantité ≤ 0 ou non entière, motif `quantite_invalide`, dans `outputs/rejets.csv`. Sources et `controles/` intacts. Sur l'échantillon je prévois 1 rejet de ce motif (`E-009-1`, quantité -1). Recopie la sortie réelle. »

2 · Demander un plan court

Plus d'un fichier touché : 3 à 5 étapes, une preuve chacune, puis vous choisissez **une** étape. Exemple : `lecture-sources` (comptes des 4 sources = les vôtres) → `formats-millimes` (`18,5` → 18500) → `rejets-motifs` (rejets par motif = votre calcul sur l'échantillon). Une branche et une PR par étape.

3 · Lire le diff, préférer les contrôles déterministes

Le diff (`git diff`, ou l'onglet de revue de l'app, Ctrl + Maj + G) montre chaque ligne ajoutée (+) ou retirée (-). Un contrôle **déterministe** donne le même résultat pour les mêmes fichiers : un compte, une somme en millimes, des rejets par motif. Exemple : après le lot 1, `OK — fact_vente : 1960 lignes`. Un diff qui touche `controles/` se refuse, même si tout est vert.



Un total peut sembler plausible et être faux : un doublon compté deux fois gonfle les ventes nettes. Le contrôle qui tranche : autant de ligne_id distincts acceptés que de lignes dans fact_vente.

03 / DIRIGER CODEX · HABITUDES 4 À 6

Les huit habitudes · 4 à 6

4 · Écrire une note courte : état, décision, preuve, prochain objectif

Une entrée de `notes/JOURNAL.md` par PR. Écrivez ce que vous avez **vu**, pas ce que Codex a promis. La note sert aussi à reprendre dans une conversation neuve (prompt 16).

notes/JOURNAL.md · exemple d'entrée

```
## 2026-11-18 — PR #6 — Charger l'étoile du lot 1
Auteur : Ines · Relecteur : Rayan · Mission : tronc-commun
- État actuel : `python -m solen initial` recrée l'entrepôt (5 tables).
- Décision : refus de relier les faits par boutique_id (proposé par Codex) ;
  on relie par boutique_key.
- Preuve : verifier --etape initial → 0 ÉCHEC ; fact_vente : 1960 lignes.
- Prochain objectif : lot 2, une boutique change de zone (SCD2).
```

5 · Distinguer tokens, contexte, effort et quota

Ce que vous observez	Le vrai problème	Le bon geste
Après trois étapes dans la même conversation, Codex oublie les millimes.	Contexte surchargé.	Note d'état, conversation neuve, AGENTS.md relu.
La réponse sur la SCD2 mélange `valid_from` et `valid_to`.	Tâche difficile, effort trop bas ?	Vérifier l'effort (High) ; redemander avec un cas daté.
Message de limite atteinte, heure de reprise affichée.	Quota.	Une conversation neuve n'y change rien : section 8.
Codex lit tout `tronc-commun/` pour une question de prix.	Trop de tokens.	Nommer le seul fichier utile : `catalogue.xlsx`.

6 · Revue lecture seule → constater → corriger → recontrôler

La revue ne modifie rien : elle **constate** au plus trois défauts avec preuve. Vous choisissez ce qui est corrigé, une demande par défaut, puis vous relancez **tous** les contrôles et testez un **cas limite**, la valeur juste à la frontière d'une règle.

Exemple Solen · cas limite SCD2

NBE passe de « Cap Bon » à « Nord-Est » le 2026-07-15 (`valid_from` inclus, `valid_to` exclu). Vente NBE du **2026-07-14** → Cap Bon ; du **2026-07-15** → Nord-Est. Si celle du 15 tombe en Cap Bon, `valid_to` est traité comme inclus : défaut établi.

03 / DIRIGER CODEX · HABITUDES 7 ET 8

Les huit habitudes · 7 et 8

7 · Utiliser les Skills locaux et AGENTS.md

`AGENTS.md`, à la racine du dépôt binôme, contient les règles que Codex lit **avant tout travail** : code seulement dans `solen/`, jamais de modification des sources, des références ni de `controles/`, millimes entiers, SQL paramétré, ni commit ni push. Les **Skills** sont des méthodes rangées dans `.agents/skills/<nom>/SKILL.md` (en-tête `name` et `description`) ; nommez-les avec `$`.

Skill fourni	Quand	Ce qu'il garantit
`$expliquer-bi`	Comprendre une notion ou un fichier	Exemple chiffré à la main d'abord ; rien n'est modifié ; une question finale.
`$revoir-bi`	Avant votre revue GitHub	Lecture seule ; au plus 3 défauts établis ; contrôles non réalisés séparés.
`$preparer-pr`	Avant `git add` / `commit` / `push`	Liste OK / À corriger / Non vérifié ; titre et description proposés.

Exemple Solen

Demandez « Quelles instructions as-tu lues ? » : la réponse doit citer `AGENTS.md`. Si Codex propose de « mettre à jour `attendu_lot1.json` pour que le contrôle passe », il contredit AGENTS.md : refusez, redemandez une correction dans `solen/`. Si `$revoir-bi` n'est pas reconnu : « Lis .agents/skills/revoir-bi/SKILL.md et applique sa méthode. »

8 · Workers parallèles : facultatifs

Les sous-agents (« subagents ») sont des agents que Codex lance en parallèle pour une partie de la tâche. Ils servent surtout au travail de **lecture** (relire sous deux angles, explorer des fichiers) et **consomment plus de tokens** qu'un agent seul. Ils ne démarrent que si vous les demandez ; leur disponibilité sur Free n'est pas garantie.

Avec sous-agents (si proposés)	Sans sous-agents (méthode de base du cours)
« Fais deux lectures indépendantes, l'une sur les données et les rejets, l'autre sur le modèle et l'historique, puis fusionne sans doublon. » Un seul auteur corrige ensuite.	Deux conversations successives avec `$revoir-bi`, une par angle. Même résultat, moins d'usage consommé.

Les huit habitudes en une ligne

Précis · plan court · diff lu et contrôle déterministe · note courte · six mots distincts · revue → correction → recontrôle + cas limite · AGENTS.md et Skills · parallèle seulement si utile.

03 / DIRIGER CODEX · PROMPTS · CADRER ET PLANIFIER

4. Les prompts prêts à adapter

Les fiches sont aussi dans `prompts/` du dépôt du cours, à copier. À utiliser dans **votre dépôt binôme**.

1. Prévoir : notez la valeur attendue avant d'envoyer
2. Adapter : remplacez chaque <...> ; une seule petite étape
3. Contrôler : relancez la commande ; comparez à votre prévision
4. Lire le diff, l'expliquer au binôme, décider

01 · Cadrer les indicateurs avec la direction

Quand : séance 1, après le mail de la direction (« les ventes montent, mais où passe la marge ? »), et au début de chaque mission du catalogue. Aucun code n'est écrit à cette étape. Tronc commun : `<dossier echantillon>` = `tronc-commun/echantillon`.

Prévoir avant d'envoyer

ouvrez l'échantillon de la mission (`echantillon/`) et calculez **à la main**, sur papier, les ventes brutes en TND. Notez votre résultat.

Prompt 01 · à adapter : remplacez chaque <...>

```
Lis mission.txt, le cahier des charges et le dictionnaire de la mission
<mission>, ainsi que les fichiers de <dossier echantillon>.
Ne modifie aucun fichier et n'écris pas de code.
Pour chaque indicateur demandé par la direction, donne en tableau :
nom, définition en une phrase, formule, unité (millimes, TND, %, quantité),
grain du fait utilisé, et le calcul sur l'échantillon ligne par ligne.
Mon calcul à la main des ventes brutes donne <ma valeur> TND : compare et
explique l'écart s'il y en a un.
Liste ensuite au plus 3 questions ambiguës à poser à la direction (ex. un
retour sans ticket compte-t-il ?), avec la conséquence de chaque réponse sur
un chiffre.
```

Ce qu'il faut vérifier dans la réponse	Pièges
• Chaque indicateur a une **unité** et une formule identique au cahier des charges (« marge sur coût d'achat », jamais « bénéfice »). • Le calcul sur l'échantillon est détaillé ligne par ligne et retombe sur `attendu.md`. • Les questions sont de vraies ambiguïtés métier, pas des questions techniques. • Notez vos décisions de cadrage dans `notes/JOURNAL.md` (rubrique Décision).	• Accepter la valeur de Codex sans avoir fait son propre calcul : vous ne saurez pas l'expliquer à l'oral. • Mélanger millimes et TND dans le même tableau (12500 « TND » au lieu de 12,500 TND).

03 / DIRIGER CODEX · PROMPTS · CADRER ET PLANIFIER

02 · Demander un plan court avant de coder

Quand : avant toute étape qui touchera plus d'un fichier (lecture des sources, nettoyage, chargement, incrément, app...). Le plan devient la liste de vos prochaines branches.

Prévoir avant d'envoyer

écrivez en une phrase le **résultat observable** que vous voulez à la fin (un fichier, un compte, un total).

Prompt 02 · à adapter : remplacez chaque <...>

```
Objectif : <résultat observable, ex. « outputs/rejets.csv existe et compte les
rejets par motif »>.
Mission : lis mission.txt et les fichiers utiles de la mission. Ne modifie
rien pour l'instant.
Propose un plan de 3 à 5 petites étapes. Pour chaque étape :
- le fichier de solen/ créé ou modifié ;
- la sortie observable ;
- la commande de contrôle à lancer et le résultat attendu ;
- un nom de branche court.
Signale ce qui est hors de mon objectif et ce que tu ne sais pas encore.
Attends mon choix de l'étape à réaliser.
```

Ce qu'il faut vérifier dans la réponse	Pièges
• Chaque étape est assez petite pour une PR relue en moins de 15 minutes. • Chaque étape a une preuve exécutable (`python -m controles.verifier …`, un compte, `pytest`), pas « vérifier que ça marche ». • Aucun fichier hors de `solen/`, `tests/`, `.github/`, `powerbi/` n'est prévu.	• Dire « oui, fais tout » : Codex écrit alors un gros diff que personne ne lit. Choisissez **une** étape. • Garder un plan dont une étape modifie `controles/` ou une source : refusez-la.

03 / DIRIGER CODEX · PROMPTS · CONSTRUIRE

03 · Réaliser une petite étape

Quand : sur une branche neuve, pour l'étape choisie dans le plan (prompt 02). Sert aussi en séance 5 pour l'application locale (`python -m solen app`). S5 : même fiche pour l'application locale, objectif « `python -m solen app` écrit `outputs/app/index.html`, autonome, filtres période et boutique/zone, indicateurs, graphique SVG ».

Prévoir avant d'envoyer

`git switch -c <branche>`. Écrivez la valeur que vous attendez (ex. « 1 482 lignes acceptées »), ou au moins son ordre de grandeur.

Prompt 03 · à adapter : remplacez chaque <...>

```
Réalise seulement l'étape <n> du plan : <rappel de l'étape en une phrase>.
Écris le code uniquement dans solen/. Ne touche ni aux sources, ni aux
références, ni à controles/.
Garde les montants en millimes entiers.
Ensuite :
1. lance <commande de contrôle> et recopie la sortie réelle ;
2. résume le diff en 3 à 6 lignes : fichiers, fonctions, rôle de chacune ;
3. indique les 3 lignes que je dois lire en priorité, avec leur numéro ;
4. je prévois <ma valeur> : dis si le résultat obtenu est égal, et sinon
   pourquoi.
Ne fais ni commit ni push.
```

Ce qu'il faut vérifier dans la réponse	Pièges
• La sortie de contrôle est **recopiée**, pas résumée par « ça passe ». Relancez la commande vous-même dans votre terminal. • `git diff` ne montre que les fichiers annoncés. Lisez les 3 lignes indiquées et expliquez-les à votre binôme. • Les montants sont des `int` (pas de `float` stocké dans l'entrepôt).	• Enchaîner trois étapes dans la même conversation sans commit entre elles : le diff devient illisible. Une étape = un commit, au plus une PR. • Croire que la commande « a été lancée » parce que Codex l'a écrite : vérifiez qu'il montre une vraie sortie, ou lancez-la.

03 / DIRIGER CODEX · PROMPTS · CONSTRUIRE

04 · Lire les sources CSV, Excel et JSON

Quand : séance 2, premier contact avec les sources de `lot1/`. Tronc commun : `<lot1>` = `tronc-commun/sources/lot1` ; comptez avec `wc -l tronc-commun/sources/lot1/ventes.csv` (l'en-tête est compté).

Prévoir avant d'envoyer

dans le terminal, `wc -l <dossier>/lot1/*.csv`. Combien de lignes de données attendez-vous (en-tête exclu) ? Ouvrez le JSON dans l'éditeur : combien de boutiques ? Notez vos deux nombres.

Prompt 04 · à adapter : remplacez chaque <...>

```
Lis mission.txt. Crée dans solen/ un module de lecture des sources de <lot1>
(le CSV de faits, le classeur Excel, le JSON) avec pandas, sans nettoyer
encore.
Lis chaque fichier avec son vrai encodage et le bon séparateur ; ne modifie
pas les fichiers sources.
Ajoute une commande temporaire ou une fonction qui affiche pour chaque
source :
nom du fichier, nombre de lignes, noms des colonnes, 3 premières lignes.
Lance-la et recopie la sortie.
J'attends <n lignes CSV> lignes dans le CSV et <n boutiques> boutiques :
compare.
Explique en 4 lignes la différence entre un DataFrame et le fichier d'origine,
et une exception qui peut arriver à la lecture (fichier absent, encodage).
```

Ce qu'il faut vérifier dans la réponse	Pièges
• Les comptes affichés sont égaux aux vôtres (sinon : en-tête ? ligne vide ? encodage ?). • Les accents s'affichent correctement (pas de Ã©). • Aucune ligne n'est déjà supprimée ou corrigée : la lecture ne nettoie pas.	• Laisser pandas convertir les montants en nombres décimaux dès la lecture : `12,500` devient faux. Mieux vaut lire en texte puis normaliser au nettoyage. • Lire l'Excel sans préciser la feuille alors que le classeur en a plusieurs.

03 / DIRIGER CODEX · PROMPTS · CONSTRUIRE

05 · Nettoyer et rejeter avec un motif

Quand : séance 2, après la lecture (prompt 04). Épisode « premier résultat rassurant… faux ». Tronc commun : comparez avec `tronc-commun/echantillon/attendu.md` **après** votre propre calcul.

Prévoir avant d'envoyer

dans l'échantillon, repérez à la main chaque ligne qui doit être gardée, normalisée ou rejetée, et son motif. Écrivez vos comptes (acceptées / rejetées par motif).

Prompt 05 · à adapter : remplacez chaque <…>

```
Applique les règles de nettoyage du cahier des charges de <mission> aux faits
de <lot1>, dans solen/ seulement. Sources intactes.
- doublon exact : garder une occurrence ;
- même identifiant avec contenu différent : rejeter les deux, motif
  id_conflit ;
- identifiant vide : id_manquant ; produit ou boutique inconnus :
  cle_inconnue ;
- quantité ≤ 0 ou non entière : quantite_invalide ;
- montants en millimes entiers (12,500 / 12.5 / 12 500 → 12500), dates en
  YYYY-MM-DD, codes en strip + majuscules.
Écris outputs/rejets.csv avec la ligne d'origine et une colonne motif.
Teste d'abord sur l'échantillon : j'attends <mes comptes>. Puis sur lot1, et
lance python -m controles.verifier --mission $(cat mission.txt) --etape
initial si ce contrôle couvre déjà les rejets.
Montre la fonction qui décide du motif et explique l'ordre des règles.
```

Ce qu'il faut vérifier dans la réponse	Pièges
• Vos comptes sur l'échantillon sont retrouvés exactement ; sinon, faites-vous expliquer **la** ligne qui diffère. • `outputs/rejets.csv` a une colonne `motif` et ne contient que les motifs prévus. • L'ordre des règles est justifié (un doublon exact n'est pas un `id_conflit`).	• Supprimer les lignes fautives sans les écrire dans `rejets.csv` : la direction ne peut plus vérifier ce qui a été écarté. • Convertir `12,500` en `12.5` puis multiplier en flottant : `12.5 * 1000` peut donner `12499.999`…. Exigez un calcul en entiers.

03 / DIRIGER CODEX · PROMPTS · CONSTRUIRE

06 · Interroger la base opérationnelle en SQL paramétré

Quand : séance 3, pour lire les retours (ou remboursements, avoirs…) dans la base SQLite opérationnelle de la mission. Tronc commun : `<retours>` = la table `retour` ; la base = `tronc-commun/sources/lot1/operations.sqlite`.

Prévoir avant d'envoyer

choisissez un mois (ex. `2026-03`). Combien de retours pensez-vous y trouver : aucun, une dizaine, une centaine ? Notez-le.

Prompt 06 · à adapter : remplacez chaque <…>

```
Dans solen/, ajoute une fonction qui lit les <retours> de la base SQLite
<chemin de la base dans lot1> avec le module sqlite3, en lecture seule.
La requête SQL doit être paramétrée avec ? (jamais de f-string ni de
concaténation) et accepter une date de début et une date de fin.
Lance-la pour le mois <AAAA-MM> et affiche : nombre de lignes, somme des
montants en millimes.
Montre la requête SQL et explique chaque clause (SELECT, FROM, WHERE, ORDER
BY).
Explique en 3 lignes pourquoi un paramètre ? protège mieux qu'une chaîne
construite.
Puis dis si une même requête avec « fin » exclue ou incluse change le résultat
sur ce mois.
```

Ce qu'il faut vérifier dans la réponse	Pièges
• La requête contient des ? et la fonction passe un tuple de paramètres. • La base est ouverte en lecture seule (rien n'est écrit dans la source). • Comparez le compte à votre estimation et refaites la requête pour un autre mois.	• Comparer des dates texte de formats différents (`04/03/2026` contre `2026-03-04`) : la comparaison SQL devient fausse. Vérifiez le format stocké. • Oublier si la borne de fin est incluse ou exclue : un retour du dernier jour disparaît ou compte deux fois d'un mois à l'autre.

03 / DIRIGER CODEX · PROMPTS · CONSTRUIRE

07 · Charger l'entrepôt en étoile (lot 1)

Quand : séance 3, quand lecture, nettoyage et SQL fonctionnent. Tronc commun : remplacez `<mission>`, `<lot1>` et vos deux prévisions `<n>` ; les noms de tables sont déjà les bons.

Prévoir avant d'envoyer

dessinez l'étoile sur papier (faits au centre, dimensions autour, clés sur les flèches). Prévoyez le nombre de lignes de `dim_boutique` et de `fact_vente` après le lot 1.

Prompt 07 · à adapter : remplacez chaque <...>

```
Lis le cahier des charges de <mission>. Dans solen/, crée une classe Entrepot
qui gère outputs/entrepot.sqlite, et la commande python -m solen initial.
initial recrée l'entrepôt depuis zéro avec <lot1> : d'abord les dimensions
(<dim_date, dim_produit, dim_boutique>), puis les faits
(<fact_vente, fact_retour>), avec des clés techniques *_key entières, des clés
métier *_id texte, des montants en millimes entiers, et une transaction pour
que tout ou rien soit écrit.
La dimension SCD2 <dim_boutique> reçoit valid_from et valid_to (NULL =
courante).
Lance python -m solen initial puis python -m controles.verifier --mission
$(cat mission.txt) --etape initial et recopie les sorties. J'attends <n>
boutiques et <n> lignes de faits : compare.
Explique le grain de chaque table de faits en une phrase.
```

`sqlite3` en ligne de commande demande le paquet du même nom (`sudo apt install -y sqlite3`). Sinon, demandez à Codex de lancer la requête **en lecture seule** et de recopier la sortie.

Ce qu'il faut vérifier dans la réponse	Pièges
• Ordre de chargement : dimensions avant faits ; aucune clé de fait sans dimension. • `sqlite3 outputs/entrepot.sqlite "SELECT COUNT(*) FROM fact_vente;"` donne le même compte que la réponse et que la référence. • Le vérificateur est réellement **OK** sur `initial` ; sinon, lisez la première ligne d'échec (prompt 10).	• Relier les faits par `boutique_id` : dès qu'une boutique aura deux versions, les ventes seront comptées deux fois. • Lancer `initial` deux fois et voir les faits doubler : `initial` doit **recréer** l'entrepôt, pas ajouter.

03 / DIRIGER CODEX · PROMPTS · CONSTRUIRE

08 · Incrément, historique SCD2 et rejeu

Quand : séance 4, épisode « changement de besoin » : le lot 2 arrive, une boutique change de zone commerciale au 2026-07-15. Tronc commun : `<lot2>` = `tronc-commun/sources/lot2` ; repérez dans son `boutiques.json` la boutique qui change de zone.

Prévoir avant d'envoyer

prévoyez trois valeurs et notez-les : (1) nombre de lignes de `dim_boutique` après le lot 2 ; (2) la zone d'une vente de cette boutique datée du 2026-07-14, puis du 2026-07-15 ; (3) le nombre de lignes de `fact_vente` après avoir lancé `increment` **deux fois**.

Prompt 08 · à adapter : remplacez chaque <...>

```
Ajoute la commande python -m solen increment qui charge <lot2> dans l'entrepôt
existant.
Règles :
- SCD2 sur <dim_boutique> : quand la zone change, fermer l'ancienne version
  (valid_to = date du changement, exclue) et créer une nouvelle version
  (valid_from = même date, incluse, valid_to NULL) ;
- un fait est rattaché à la version valable à SA date ; ne jamais modifier les
  faits anciens ;
- rejeu : relancer le même lot n'ajoute aucune ligne (clé métier unique,
  insertion idempotente).
Lance : initial, increment, increment, puis les vérifications --etape
increment et --etape rejeu, et recopie les sorties. Montre la requête qui
choisit la version de boutique pour une date.
Mes prévisions : <1>, <2>, <3>. Compare chacune avec le résultat réel.
```

Ce qu'il faut vérifier dans la réponse	Pièges
• La version choisie utilise `valid_from <= date AND (valid_to IS NULL OR date < valid_to)`. • Les comptes avant et après le deuxième `increment` sont **identiques**. • Une vente ancienne de la boutique reste dans l'ancienne zone (contrôle à la main avec `sqlite3`).	• Mettre à jour la zone dans la ligne existante (`UPDATE … SET zone`) : tout l'historique bascule dans la nouvelle zone, le passé est réécrit. • Prendre `valid_to` inclus : la vente du jour du changement est rattachée à deux versions, ou à la mauvaise.

03 / DIRIGER CODEX · PROMPTS · CONSTRUIRE

09 · Tests automatiques et CI

Quand : séance 4, quand `initial`, `increment` et `export` existent. Objectif : une CI verte sur GitHub à chaque PR. Valeur actuelle de `ETAPE_CI` : dans `.github/workflows/controles.yml` (au départ `aucune`).

Prévoir avant d'envoyer

choisissez une règle (ex. « une quantité de -2 est rejetée »). Prévoyez : si Codex casse cette règle dans le code, quel test doit devenir rouge ?

Prompt 09 · à adapter : remplacez chaque <...>

```
Dans tests/ (pas dans controles/), écris 3 à 5 tests pytest courts pour nos
règles :
<règle 1>, <règle 2>, le rejeu, et la version SCD2 à la date frontière <date>.
Chaque test a un nom en français qui dit la règle, et utilise un petit jeu de
données construit dans le test (pas les sources complètes).
Lance pytest -q et recopie la sortie.
Puis, sans le garder, montre-moi quel test échouerait si on supprimait la
règle <règle 1>, et pourquoi.
Ensuite mets à jour .github/workflows/controles.yml : passe ETAPE_CI de
<valeur actuelle> à <nouvelle valeur>, et explique en 5 lignes ce que la CI va
exécuter. Ne fais ni commit ni push.
```

Ce qu'il faut vérifier dans la réponse	Pièges
• `pytest -q` passe en local avant de pousser. • Le diff de `controles.yml` ne change que ce que vous avez demandé ; aucun secret, aucune clé d'IA, aucun service externe n'y apparaît. • Après le push, onglet **Actions** de la PR : la CI s'exécute ; ouvrez une étape et lisez sa sortie.	• Écrire des tests qui recopient le résultat du code (`assert total == total`) : ils ne peuvent jamais échouer. Un bon test part d'un exemple calculé à la main. • Passer `ETAPE_CI` à `tout` alors que l'export n'existe pas encore : la CI est rouge pour une raison attendue, et l'équipe s'habitue à ignorer le rouge.

03 / DIRIGER CODEX · PROMPTS · REVUE ET PULL REQUEST

11 · Revue Codex en lecture seule

Quand : le relecteur, après `git fetch` puis `git switch <branche>`, avant sa propre revue écrite sur GitHub. En séance 4, sur la branche qui contient le défaut planté.

Prévoir avant d'envoyer

relancez les contrôles de l'étape. Lisez le diff vous-même 5 minutes et notez **un** point qui vous paraît douteux.

Prompt 11 · à adapter : remplacez chaque <...>

```
Utilise $revoir-bi. Branche : <branche>. Mission : lis mission.txt.
Relis en lecture seule le diff de cette branche par rapport à main : ne
modifie rien, ne lance aucun chargement ni export.
Angle de la revue : <données et rejets | totaux | modèle et clés | SCD2 |
rejeu>.
Rapporte au plus 3 défauts établis (fichier:ligne, règle violée, exemple
observable, correction proposée), puis les contrôles non réalisés.
J'ai moi-même un doute sur : <mon point>. Dis s'il est fondé, avec une preuve.
```

Sous-agents : si votre client en propose et que vous le demandez, ajoutez « Fais deux lectures indépendantes, l'une sur les données, l'autre sur le modèle et l'historique, puis fusionne sans doublon. » Sinon, deux conversations successives, une par angle. `$revoir-bi` non reconnu : « Lis .agents/skills/revoir-bi/SKILL.md et applique sa méthode. »

Ce qu'il faut vérifier dans la réponse	Pièges
• Chaque défaut a un **exemple observable** (une valeur, une ligne, un compte) que vous pouvez reproduire. • Les « contrôles non réalisés » sont séparés des défauts. • `git status` est inchangé après la revue : rien n'a été modifié.	• Coller la conclusion de Codex comme revue GitHub : votre revue écrite doit dire ce que **vous** avez vérifié et reproduit. • Prendre « aucun défaut établi » pour « tout est juste » : ce n'est que le résultat d'une lecture sous un angle.

03 / DIRIGER CODEX · PROMPTS · REVUE ET PULL REQUEST

12 · Corriger après revue, puis faire relire

Quand : l'auteur, quand la PR a reçu « Request changes » ou des constats de revue.

Prévoir avant d'envoyer

pour chaque constat, prévoyez : défaut réel ou faux constat ? Quelle valeur changera après correction ?

Prompt 12 · corriger (auteur)

```
Voici les constats de revue reçus sur ma PR <n> :
<coller les constats>
Pour chacun, vérifie d'abord contre le code actuel : établi, non établi, ou
non vérifiable, avec une preuve observable.
Corrige seulement les défauts établis, dans solen/ (ou tests/), sans toucher
aux sources, aux références ni à controles/.
Lance <contrôles de l'étape> et pytest -q, recopie les sorties.
Explique pour chaque constat rejeté pourquoi, en une phrase.
Je prévois que <valeur> passera de <avant> à <après> : compare.
Ne fais ni commit ni push.
```

Prompt 12 · deuxième revue, nouvelle conversation (ou relecteur)

```
Utilise $revoir-bi sur la version corrigée de la branche <branche>, en lecture
seule.
Pour chaque constat initial : corrigé avec preuve, encore présent, ou non
vérifié.
Cherche aussi une erreur introduite par la correction. Ne modifie rien.
```

Ce qu'il faut vérifier dans la réponse	Pièges
• Une correction = un défaut ; le diff ne contient pas de « nettoyage » non demandé. • Le contrôle qui échouait passe maintenant, et les autres restent verts. • Répondez à chaque commentaire GitHub (« corrigé dans <commit> » ou « non retenu parce que… ») avant de redemander une revue.	• Corriger en modifiant le contrôle ou la valeur attendue : c'est interdit, et le relecteur le verra dans le diff. • Pousser la correction sans relancer **tous** les contrôles : une correction en casse souvent une autre (ex. le rejeu).

03 / DIRIGER CODEX · PROMPTS · REVUE ET PULL REQUEST

13 · Préparer la pull request

Quand : l'auteur, quand l'étape est terminée et avant `git add` / `git commit` / `git push`.

Prévoir avant d'envoyer

écrivez, sans aide, la phrase « Cette PR permet de … » et la commande qui le prouve.

Prompt 13 · préparer

```
Utilise $preparer-pr pour la branche <branche>.
Étape à contrôler : <initial | increment | rejeu | export | tout>.
Relecteur : <prénom et compte GitHub du partenaire>.
Ma phrase : « Cette PR permet de <…> », preuve : <commande>.
Fais la liste de contrôle, puis propose un titre et une description en
français.
Ne commite pas, ne pousse pas, n'ouvre pas la PR.
```

Terminal · vous-même, après la liste de contrôle

```
git status
git add <fichiers listés>
git commit -m "<titre proposé>"
git push -u origin <branche>
```

Sur GitHub : **Compare & pull request**, collez la description, désignez le relecteur dans **Reviewers**, puis **Create pull request**. `$preparer-pr` non reconnu : « Lis .agents/skills/preparer-pr/SKILL.md et applique sa méthode. »

Ce qu'il faut vérifier dans la réponse	Pièges
• Toutes les lignes de la liste sont OK, ou vous savez pourquoi l'une ne l'est pas. • Aucun fichier `outputs/`, `.venv/` ou secret dans `git status` avant `git add`. • L'entrée de `notes/JOURNAL.md` contient une preuve réelle (commande + résultat).	• `git add .` sans relire `git status` : un fichier inattendu part dans la PR. • Ouvrir la PR depuis `main` : il n'y a rien à comparer, ou pire, on a commité sur `main` directement.

03 / DIRIGER CODEX · PROMPTS · QUAND ÇA BLOQUE

10 · Comprendre une erreur avant de la corriger

Quand : une commande échoue (Python, vérificateur, pytest, Git, CI).

Prévoir avant d'envoyer

recopiez le message **exact**, en entier, depuis le terminal ou depuis l'étape de la CI. Lisez la dernière ligne : quelle est votre hypothèse ?

Prompt 10 · à adapter : remplacez chaque <...>

```
J'ai lancé : <commande exacte>
Depuis le dossier : <résultat de pwd>
J'attendais : <résultat attendu>
J'ai obtenu ce message exact :
<coller le message complet, sans le raccourcir>
Mon hypothèse : <une phrase>.
Avant de modifier quoi que ce soit : explique ce que dit la dernière ligne,
quel fichier et quelle ligne sont en cause, et si mon hypothèse est juste.
Propose ensuite la plus petite correction, dans solen/ seulement, et la
commande qui prouvera qu'elle marche. Attends mon accord avant de modifier.
```

Ce qu'il faut vérifier dans la réponse	Pièges
• Le diagnostic cite un fichier et une ligne présents dans **votre** message. • La correction proposée ne touche ni `controles/`, ni les sources, ni les références. • Après correction, relancez exactement la même commande : même dossier, même venv.	• Résumer l'erreur (« ça marche pas ») au lieu de la coller : Codex devine, et corrige autre chose. • Oublier d'activer le venv (`source .venv/bin/activate`) : `ModuleNotFoundError` ne vient alors pas du code.

03 / DIRIGER CODEX · PROMPTS · QUAND ÇA BLOQUE

15 · Résoudre un conflit Git

Quand : `git pull` ou un **Merge** affiche `CONFLICT`. En séance 4, le conflit est provoqué volontairement : A et B modifient la même ligne de `notes/JOURNAL.md` dans deux branches.

Prévoir avant d'envoyer

tapez `git status`. Quels fichiers sont « both modified » ? Ouvrez-en un et repérez `<<<<<<<`, `=======`, `>>>>>>>`. Qu'est-ce que chacun des deux blocs voulait dire ?

Prompt 15 · à adapter : remplacez chaque <...>

```
J'ai un conflit Git. Voici la sortie exacte de git status :
<coller>
Explique en mots simples ce qui s'est passé (quelles deux versions se
rencontrent).
Pour chaque fichier en conflit, montre les deux blocs et dis ce que chacun
voulait garder.
Propose une résolution qui garde l'intention des deux, sans perdre de contrôle
ni de règle.
Ne modifie un fichier qu'après mon accord, et seulement les fichiers en
conflit.
Ne lance ni git add, ni git commit, ni git push : je les fais moi-même.
Après résolution, indique les contrôles à relancer.
```

Puis, vous-même : relire le fichier (plus aucun marqueur), relancer les contrôles, `git add <fichier>`, `git commit` (message proposé par Git), `git push`.

Ce que montre un conflit dans notes/JOURNAL.md : en haut votre version, en bas l'autre

```
<<<<<<< HEAD
- Prochain objectif : charger le lot 2 (SCD2 sur dim_boutique).
=======
- Prochain objectif : écrire les tests du rejeu.
>>>>>>> tests-rejeu
```

Ce qu'il faut vérifier dans la réponse	Pièges
• La résolution proposée contient les deux intentions, ou explique clairement pourquoi l'une est abandonnée — c'est **votre** décision, pas celle de Codex. • `grep -n "<<<<<<<|>>>>>>>" <fichier>` ne trouve plus rien. • Les contrôles passent après la résolution, pas seulement avant.	• Garder « ma version » d'un clic sans lire l'autre : le travail du binôme disparaît. • Laisser un marqueur `=======` dans un fichier Python : le code ne s'exécute plus.

03 / DIRIGER CODEX · PROMPTS · EXPLIQUER ET REPRENDRE

14 · Expliquer un résultat (ligne source → graphique)

Quand : quand un chiffre vous surprend, et pour préparer la soutenance (suivre une ligne de la source jusqu'au graphique, expliquer un rejet, une version historique, un indicateur et son unité).

Prévoir avant d'envoyer

choisissez une ligne précise d'une source (son identifiant) et prévoyez où elle finit : rejetée ? dans quel fait, quelle zone, quel mois ?

Prompt 14 · à adapter : remplacez chaque <...>

```
Utilise $expliquer-bi. Ne modifie rien.
Suis la ligne <identifiant> de <fichier source> à travers notre pipeline :
lecture, nettoyage (gardée, normalisée ou rejetée et pourquoi), clé de date,
de produit, de boutique (quelle version SCD2 et pourquoi), ligne de fait,
export CSV, et contribution à l'indicateur <indicateur> dans le graphique
<graphique>.
Donne chaque valeur avec son unité, et le fichier:ligne du code qui la
produit.
Ma prévision : <où elle finit>. Dis si j'ai raison.
Termine par une question pour vérifier que j'ai compris.
```

Ce qu'il faut vérifier dans la réponse	Pièges
• Chaque étape cite une valeur que vous pouvez retrouver (dans la source, `rejets.csv`, `sqlite3`, le CSV exporté, Power BI). • Retrouvez vous-même au moins une de ces valeurs ; répondez à la question finale **sans** relire la réponse.	• Choisir une ligne « facile » : prenez aussi une ligne rejetée et une ligne de la boutique qui change de zone, ce sont les questions de soutenance. • Réciter l'explication de Codex : reformulez avec vos mots et votre exemple.

03 / DIRIGER CODEX · PROMPTS · EXPLIQUER ET REPRENDRE

16 · Faire une note d'état et reprendre

Quand : en fin de séance, quand la conversation devient longue, ou quand le partenaire reprend la main. Le **contexte** est ce que la conversation contient ; le **quota** est la limite d'usage du compte. Une conversation neuve ne remet pas le quota à zéro, mais elle évite que Codex oublie les règles.

Prévoir avant d'envoyer

sans regarder, écrivez la dernière commande réussie et la prochaine petite étape. Vous comparerez avec la note.

Prompt 16 · fin de séance

```
Rédige une note d'état de 8 lignes maximum, que je copierai moi-même. Ne
modifie aucun fichier.
Contenu : mission (mission.txt), branche actuelle, ce qui fonctionne,
commandes réellement exécutées et leur résultat, défaut encore ouvert,
prochaine petite étape et sa preuve de réussite.
Sépare clairement : réalisé / à faire / non vérifié. Aucun identifiant de
compte.
```

Prompt 16 · reprise, dans une nouvelle conversation

```
Je reprends le dépôt binôme. Lis AGENTS.md, mission.txt et notes/JOURNAL.md.
Voici ma note d'état :
<coller la note>
Vérifie les fichiers cités et l'état Git (branche, fichiers modifiés) ;
ne suppose pas qu'une commande a été exécutée.
Propose la prochaine petite action et sa preuve. Ne modifie rien pour
l'instant.
```

Vérifiez une ligne de la note contre le terminal ou un fichier, puis reportez l'essentiel dans `notes/JOURNAL.md` si une PR est concernée.

Ce qu'il faut vérifier dans la réponse	Pièges
• La note distingue ce qui a été **exécuté** de ce qui a été **proposé**. • À la reprise, Codex constate l'état réel (branche, `git status`) au lieu de répéter la note. • Relevez ce qui est affiché : modèle, effort, indication de contexte ou d'usage (« non affiché » sinon). Ne demandez pas à Codex de deviner son quota.	• Recoller toute l'ancienne conversation : on retrouve le même contexte surchargé. • Quota atteint, abandonner la séance : continuez les actions locales (lire le diff, relancer un contrôle, calculer à la main, préparer la prochaine demande) et attendez le reset affiché, sans achat.

03 / DIRIGER CODEX · PROMPTS · ALLER PLUS LOIN

17 · Créer et essayer un Skill local

Quand : séance 4, après avoir utilisé `$revoir-bi` au moins une fois. Un **Skill** est une méthode réutilisable rangée dans `.agents/skills/<nom>/SKILL.md`, avec un en-tête `name` et `description`. Il n'ajoute ni modèle, ni outil, ni quota.

Prévoir avant d'envoyer

lisez `.agents/skills/revoir-bi/SKILL.md`. Choisissez **une** règle que vous voudriez toujours voir vérifiée sur votre mission (ex. « contrôler une vente la veille et le jour du changement SCD2 »). Prévoyez comment la réponse changera avec cette règle.

Prompt 17 · créer le Skill

```
Crée .agents/skills/<ma-revue-mission>/SKILL.md, en partant de
.agents/skills/revoir-bi/SKILL.md.
En-tête : name: <ma-revue-mission>, et une description d'une phrase qui dit
quand l'utiliser.
Garde la lecture seule et la limite de 3 défauts établis.
Ajoute cette règle précise : <ma règle>.
Ne modifie pas les Skills fournis. Montre le fichier complet.
```

Prompt 17 · l'essayer, dans une nouvelle conversation

```
Utilise $<ma-revue-mission> pour relire <angle> sur la branche <branche>.
Lecture seule.
```

Skill non proposé : redémarrez l'application et rouvrez le dossier du dépôt ; sinon « Lis .agents/skills/<ma-revue-mission>/SKILL.md et applique sa méthode. » Notez la route qui a marché : la seconde prouve qu'on réutilise une méthode écrite, pas que le Skill est découvert automatiquement.

Ce qu'il faut vérifier dans la réponse	Pièges
• La réponse applique **votre** règle de façon visible (ex. les deux dates sont contrôlées et la version retenue est expliquée). • Le Skill est versionné : il passe par une branche et une PR comme le reste.	• Une règle vague (« vérifie bien tout ») : elle ne change rien à la réponse. Une bonne règle est observable. • Croire qu'un Skill remplace les contrôles : comparez toujours avec `python -m controles.verifier`.

03 / DIRIGER CODEX · PROMPTS · ALLER PLUS LOIN

18 · Réaliser une variation métier (mission du catalogue)

Quand : sur votre mission du catalogue (`mission.txt` = `Mxx`), quand la direction fictive demande une **variante** de la fiche mission (nouvel indicateur, nouvelle règle, nouvelle source ou changement historique). Bon entraînement pour la question « résoudre une variation avec Codex et la contrôler ».

Prévoir avant d'envoyer

lisez la variante dans `catalogue/<Mxx>/fiche_mission.md`. Prévoyez : quelles tables changent ? Quel indicateur bouge, dans quel sens, et environ de combien sur l'échantillon ?

Prompt 18 · à adapter : remplacez chaque <...>

```
Mission <Mxx> : lis mission.txt, la fiche mission, le cahier des charges et le
dictionnaire.
La direction demande la variante <n> : <recopier la variante>.
Ne modifie rien. Donne :
1. la règle métier exacte et ce qui reste ambigu (question à poser à la
   direction) ;
2. l'impact : fichiers de solen/, tables, colonnes, indicateurs, exports, DAX,
   app ;
3. le calcul à la main sur l'échantillon, avant / après la variante ;
4. un plan de 2 à 4 petites étapes, chacune avec sa preuve ;
5. le test pytest qui prouvera la nouvelle règle.
Ma prévision : <tables touchées, sens et ordre de grandeur>. Compare.
```

Ensuite : prompt **03** pour chaque étape du plan, sur une branche dédiée, puis **09** pour le test et **13** pour la PR.

Ce qu'il faut vérifier dans la réponse	Pièges
• L'ambiguïté est tranchée par **vous** (et notée dans `notes/JOURNAL.md`, rubrique Décision), pas par Codex. • Les contrôles de référence de la mission restent verts : une variante ajoute, elle ne casse pas l'existant. • Le nouveau chiffre est vérifié à la main sur l'échantillon.	• Coder la variante avant d'avoir mesuré son impact : on oublie l'export, le DAX ou l'app, et le rapport contredit l'entrepôt. • Modifier la valeur de référence pour qu'elle colle au nouveau calcul : la référence décrit la mission de base ; la variante se prouve par **son** test.

03 / DIRIGER CODEX · LIRE LE CODE

5. Lire un diff et un fichier Python sans savoir coder

Vous n'écrivez pas de code, mais vous devez savoir **où regarder** et expliquer à l'oral une ligne importante. Quatre formes suffisent pour se repérer dans ce que Codex produit.

Forme	À quoi elle ressemble	Ce qu'elle veut dire
Module	Un fichier `solen/conversions.py` ; ailleurs : `from .conversions import texte_vers_millimes`	Un fichier Python qui regroupe des fonctions d'un même sujet. `import` = « j'utilise ce qui est défini là-bas ».
Fonction	`def texte_vers_millimes(texte) -> int:` puis un bloc décalé à droite, et `return …`	Une petite machine nommée : elle reçoit `texte`, renvoie un entier (`-> int`). Le texte entre `"""` dit ce qu'elle fait.
Exception	`raise RejetLigne("quantite_invalide", …)` ; ailleurs `try:` … `except RejetLigne as e:`	`raise` signale un problème et interrompt le calcul ; `except` l'attrape pour faire autre chose (écrire la ligne dans `rejets.csv`).
Classe	`class Entrepot:` puis des fonctions `def charger_faits(self, lignes):`	Un objet qui regroupe des données (chemin, connexion) et des actions sur elles ; `self` = l'objet lui-même.

Lire un diff

git diff · exemple

```
diff --git a/solen/conversions.py b/solen/conversions.py
@@ -12,7 +12,8 @@ def texte_vers_quantite(texte) -> int:
     t = str(texte).strip()
-    if int(t) < 0:
+    if not re.fullmatch(r"\d+", t) or int(t) <= 0:
         raise RejetLigne("quantite_invalide", f"quantité {texte!r}")
```

- `diff --git a/… b/…` : le fichier modifié (a = avant, b = après).
- `@@ -12,7 +12,8 @@` : autour de la ligne 12 ; la fonction concernée est rappelée à droite.
- `-` ligne retirée, `+` ligne ajoutée ; sans signe : contexte. Ici, une quantité `0` ou `1.5` devient aussi un rejet : prévoyez combien de rejets `quantite_invalide` en plus sur l'échantillon, puis vérifiez.

03 / DIRIGER CODEX · LIRE LE CODE

Où regarder dans un fichier produit par Codex

Vous cherchez	Indices dans le code	Exemple Solen
Où le fichier est **lu**	`pd.read_csv(`, `pd.read_excel(`, `json.load(`, `sqlite3.connect(` ; `sep=`, `encoding=`, `sheet_name=`, `dtype=str`	`ventes.csv` : `;`, UTF-8 ; `catalogue.xlsx` : feuille `Catalogue`, en-têtes ligne 3
Où une ligne est **rejetée**	`raise RejetLigne(` ; écriture dans `rejets.csv` avec un `motif`	`E-006-1` deux fois, 2 puis 4 savons → les deux en `id_conflit`
Où les **millimes** sont calculés	Une fonction de conversion ; `* 1000` sur un `float` est suspect	`18,5` = `18.500` = 18500 ; `1 240,000` = 1240000
Où la **version de boutique** est choisie	`valid_from <= ?` et `valid_to IS NULL OR ? < valid_to`	NBE le 2026-07-14 → Cap Bon ; le 2026-07-15 → Nord-Est

Code produit par Codex · solen/conversions.py (extrait)

```
def texte_vers_millimes(texte) -> int:
    """« 18,500 », « 18.5 », « 1 240,000 » → millimes entiers."""
    t = str(texte).strip().replace(" ", "").replace(",", ".")
    if not re.fullmatch(r"\d+(\.\d+)?", t):
        raise RejetLigne("format_invalide", f"montant illisible {texte!r}")
    entier, _, dec = t.partition(".")
    return int(entier) * 1000 + int((dec + "000")[:3])
```

Ligne	Ce qu'elle fait, en mots
1-3	Reçoit un texte, renvoie un entier ; enlève les espaces (`1 240,000` → `1240,000`), virgule → point.
4-5	Si le texte n'a pas la forme « chiffres, éventuellement un point et des chiffres », la ligne est rejetée avec un motif.
6-7	Coupe au point (`18` et `5`), complète à trois décimales (`500`) : 18 × 1000 + 500 = 18500. Aucun `float`, donc aucune erreur d'arrondi.

À vous

Prévoyez sans exécuter : `"3,9"` ? `"14.5"` ? `"12,5 TND"` ? Puis faites lancer ces trois cas par Codex (`$expliquer-bi`) et comparez. Pourquoi le troisième est-il rejeté ?

03 / DIRIGER CODEX · REVUE

6. La revue à deux

Une PR n'entre dans `main` qu'après la revue de **l'autre** membre du binôme. L'auteur ne peut pas approuver sa propre PR (GitHub ne le permet pas) et ne la fusionne pas sans approbation. Les rôles alternent à chaque PR.

1. Récupérer : git fetch puis git switch <branche>
2. Relancer les contrôles ; tester un cas limite
3. Codex en lecture seule : $revoir-bi (prompt 11)
4. Écrire SA revue : Files changed → Review changes

Sur GitHub	Ce que vous faites
Files changed	Lire chaque fichier modifié ; sur une ligne, **+** pour un commentaire précis (fichier, ligne, exemple).
Review changes	Résumer : ce que vous avez vérifié (commande + résultat) et ce que vous n'avez pas vérifié.
Approve / **Request changes**	Approve seulement si les contrôles sont verts chez vous **et** dans la CI, et si vous savez expliquer la ligne principale. Sinon Request changes, avec ce qu'il faut corriger. On ne fusionne jamais sur rouge.
Submit review	L'auteur corrige sur la même branche (prompt 12) et répond sous chaque commentaire : « corrigé dans `<commit>` » ou « non retenu parce que… » avec la preuve.

Revue faible	Revue utile
« OK, Codex dit qu'il n'y a aucun défaut. »	« `solen/historique.py:42` : `valid_to` comparé avec <=. Reproduit : une vente NBE du 2026-07-15 tombe en Cap Bon au lieu de Nord-Est ; `--etape increment` : « 1 faits mal rattachés ». Non vérifié : le rejeu. → Request changes. »

Le « OK » de Codex n'est pas votre jugement

« Aucun défaut établi par cette lecture » veut dire : rien trouvé sous **cet** angle. Votre revue dit ce que **vous** avez reproduit. Ne collez jamais la réponse de Codex comme revue GitHub ; citez au moins une valeur vue vous-même.

03 / DIRIGER CODEX · GIT

7. Git au quotidien

Git enregistre l'histoire de vos fichiers sur votre PC ; GitHub héberge la copie partagée. La plupart des erreurs viennent de trois questions oubliées : **où suis-je** (`pwd`), **sur quelle branche** (`git status`, `git branch`), **qu'est-ce qui a changé** (`git diff`, `git log --oneline -5`) ? Lisez avant d'agir.



Trois emplacements : vos deux copies locales et GitHub. commit enregistre localement, push envoie, pull récupère, la PR propose de fusionner une branche dans main.



Le cycle complet d'une pull request à deux, de `main` à jour au `pull` des deux copies ; puis les rôles s'inversent.

03 / DIRIGER CODEX · GIT

Le cycle, en liste de contrôle

#	Qui	Geste	Signe de réussite
1	A	`cd ~/solen-bi` · `source .venv/bin/activate`	`(.venv)` dans l'invite
2	A	`git switch main` · `git pull`	« Already up to date » ou commits reçus
3	A	`git switch -c <branche>` (ex. `rejets-motifs`)	« Switched to a new branch »
4	A	Demande à Codex : une petite étape, une prévision	Sortie réelle recopiée
5	A	Contrôles : `python -m controles.verifier …`, `pytest controles`	Résultat = votre prévision
6	A	`git status` · `git diff` · `$preparer-pr`	Rien dans `outputs/` ni `.venv/`
7	A	`git add <fichiers>` · `git commit -m "<message>"`	« nothing to commit »
8	A	`git push -u origin <branche>`	Lien « Create a pull request »

#	Qui	Geste	Signe de réussite
9	A	GitHub : **Compare & pull request** → relecteur B → **Create pull request**	PR ouverte, CI lancée
10	B	`git fetch` · `git switch <branche>`	* sur la branche (`git branch`)
11	B	Contrôles + `$revoir-bi` en lecture seule + sa propre lecture	`git status` inchangé
12	B	**Files changed → Review changes → Approve / Request changes**	Revue avec une valeur reproduite
13	A	Corrections sur la même branche : `git commit` · `git push`	La PR se met à jour
14	B	CI verte + approbation → **Merge pull request → Confirm merge**	« Merged »
15	A et B	`git switch main && git pull` · `git branch -d <branche>`	Même `git log --oneline -1`

A est l'auteur de cette PR, B le relecteur ; à la PR suivante, les rôles s'inversent.

03 / DIRIGER CODEX · GIT · DÉPANNAGE

Dépanner Git sans rien casser

Symptôme	Cause	Réparer
`fatal: not a git repository`	**Mauvais dossier** (~ ou le dépôt du cours `~/cours-bi`).	`cd ~/solen-bi`, puis `git status`.
Commit fait alors que `git status` dit « On branch main ».	**Mauvaise branche** : `git switch -c` oublié.	`git switch -c <branche>` (le commit suit), `git push -u origin <branche>`, PR. Après fusion : `git switch main && git pull`.
`Your local changes … would be overwritten by checkout`	**Changement non commité** au moment de changer de branche.	Voulu : commit sur la bonne branche. Sinon : `git stash`, changer de branche, `git stash pop`.
Push refusé : `protected branch`	Push direct sur `main`, protégée.	Normal : branche + PR.
Push refusé : `fetch first` / `rejected`	**Push refusé** : GitHub a des commits que vous n'avez pas.	`git pull` sur la branche, contrôles, `git push`. Jamais `--force`.
PR « out-of-date » ; votre `main` local n'a pas la dernière fusion.	**main en retard**.	`git switch main && git pull` ; sur la branche : `git fetch`, `git merge origin/main`, contrôles, `git push`.
Croix rouge sur la PR	**PR avec contrôle rouge** : la CI a échoué.	**Details**, lire la fin ; relancer la même commande chez soi ; prompt 10 ; corriger le code, jamais les contrôles ni la référence.
`CONFLICT (content): Merge conflict in …`	**Conflit** : A et B ont modifié les mêmes lignes.	Prompt 15 : lire les deux versions, décider à deux, retirer `<<<<<<< ======= >>>>>>>`, contrôles, `git add`, `git commit`. Abandonner : `git merge --abort`.
`Not possible to fast-forward`	Histoires divergentes.	Ne rien forcer ; montrer `git status` et `git log --oneline -5`.

Interdits

`git push --force`, `git reset --hard`, supprimer `.git`, fusionner sa propre PR sans approbation : ils effacent du travail ou contournent la revue. Avant toute réparation : `pwd`, `git status`, `git log --oneline -5` ; en cas de doute, montrez-les au binôme ou au professeur.

03 / DIRIGER CODEX · SANS CODEX

8. Quand Codex est indisponible ou le quota atteint

Cela arrivera : message de limite atteinte avec une heure de reprise, application indisponible, modèle pas encore visible sur votre compte (déploiement progressif). Votre travail ne s'arrête pas : la moitié du métier se fait sans IA.

Vous continuez…	Exemple Solen
Git : commiter ce qui est contrôlé, pousser, ouvrir la PR	L'étape `rejets-motifs` passe le contrôle : `git add`, `git commit`, `git push`, PR avec la preuve.
La revue du binôme sur GitHub	Relire sa PR, relancer ses contrôles, écrire une revue précise (section 6).
L'interprétation	Lire `outputs/export/manifest.json` ou les visuels de `reference/visuels/` ; préparer une phrase pour la direction : quelle boutique, quel mois, quelle marge.
Lire le diff et l'expliquer	`git diff main...` ; repérer où les millimes sont calculés ; l'expliquer à voix haute.
Calculer à la main	Recalculer sur l'échantillon ventes nettes et taux de retour ; comparer à `attendu.md`.
Préparer la prochaine demande	Note d'état (prompt 16) écrite par vous ; prompt suivant adapté, sans <…>, avec votre prévision.

Le point de reprise fourni

Perdus, ou un fichier fourni abîmé ? Vous repartez de deux choses : le dossier `modele-binome/` **du dépôt du cours** (sources, références, `controles/`, `AGENTS.md` et Skills d'origine) et votre `notes/JOURNAL.md` (dernière preuve réelle, prochain objectif). Un fichier fourni abîmé se remplace par sa copie d'origine, sur une branche, avec une PR relue.

Si votre écran est différent

Luna absent, effort grisé, Advanced introuvable : notez ce qui s'affiche (date, capture), utilisez ce qui est proposé, prévenez le professeur. Ne payez rien ; pas le compte d'un autre.

Jamais

Écrire ou corriger du code à la main pour « avancer quand même ». Acheter une offre ou changer de plan. Utiliser le compte Codex d'une autre personne ou coller un jeton. Attendez le reset affiché, notez l'heure dans le journal, prévenez le professeur si le blocage dure.

03 / DIRIGER CODEX · ERREURS

9. Erreurs Python et contrôles rouges les plus fréquents

Un message d'erreur Python (« traceback ») se lit **de bas en haut** : la dernière ligne dit quoi, les lignes `File "…", line …` au-dessus disent où. Recopiez-le en entier dans le prompt 10.

Terminal · un traceback typique

```
Traceback (most recent call last):
  File "/home/ines/solen-bi/solen/lecture.py", line 27, in lire_ventes
    return pd.read_csv(chemin, sep=";", dtype=str, encoding="utf-8")
FileNotFoundError: [Errno 2] No such file or directory:
'tronc-commun/lot1/ventes.csv'
```

Dernière ligne	Ce que ça veut dire	Ce qu'on demande à Codex
`FileNotFoundError: … No such file or directory`	Fichier absent à ce chemin. Vérifiez `pwd` (= `~/solen-bi`) et le vrai chemin : `ls tronc-commun/sources/lot1`.	Construire le chemin depuis `mission.txt` (`tronc-commun/sources/lot1/ventes.csv`), pas en dur.
`UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe9`	Le fichier n'est pas en UTF-8 : `0xe9` est un « é » en `cp1252` (Windows). Prévu au lot 2 de certaines missions.	Lire **ce** fichier en `cp1252`, sans modifier la source ; vérifier qu'« espèces » s'affiche bien.
`ValueError: could not convert string to float: '12,500'`	Conversion directe d'un montant écrit à la française.	Convertir en millimes entiers avec une fonction testée sur `12,500`, `12.5`, `12 500`.
`sqlite3.IntegrityError: FOREIGN KEY constraint failed`	Un fait pointe vers une clé de dimension absente : dimensions chargées après les faits, ou clé inconnue non rejetée.	Charger les dimensions d'abord ; rejeter la ligne en `cle_inconnue` au lieu de l'insérer.
`sqlite3.IntegrityError: UNIQUE constraint failed: fact_vente.ligne_id`	Une ligne déjà chargée est réinsérée (deuxième `increment`).	Rendre l'insertion idempotente, sans supprimer de faits existants.
`ModuleNotFoundError: No module named 'pandas'`	Venv inactif : le code n'y est pour rien.	Rien : `source .venv/bin/activate`, puis relancer.

03 / DIRIGER CODEX · ERREURS

Lire un contrôle rouge

Le vérificateur ne lit pas votre code : il compare vos sorties (`outputs/`) aux valeurs de référence. Chaque ÉCHEC a la même forme : **ce qui est contrôlé : attendu X, observé Y**. L'écart entre X et Y est votre premier indice.

Terminal · python -m controles.verifier --mission tronc-commun --etape increment

```
== Étape increment : comparaison avec attendu_lot2.json ==
OK — dim_boutique : 4 lignes
ÉCHEC — fact_retour : attendu 99 lignes, observé 98
ÉCHEC — rejets lot2 : cle_inconnue, attendu 4, observé 8
ÉCHEC — fact_vente : attendu chaque fait relié à la version valable à sa
date, observé 1 faits mal rattachés

Résultat : 58 OK, 3 ÉCHEC.
```

Ligne ÉCHEC	Hypothèse à vérifier	Votre prévision après correction
`fact_retour : attendu 99, observé 98`	Un retour valide du lot 2 a été perdu (rejeté à tort, ou date hors bornes).	99 lignes, aucun nouveau rejet.
`rejets lot2 : cle_inconnue, attendu 4, observé 8`	Exactement le double : les rejets du lot 2 ont été écrits deux fois.	4, même après un deuxième `increment`.
`… observé 1 faits mal rattachés`	Une vente du 2026-07-15 pointe vers l'ancienne version : `valid_to` traité comme inclus.	La vente NBE du 15 juillet passe en Nord-Est.
`fact_vente : attendu des entiers …`	Un montant stocké en `float` (TND) au lieu de millimes entiers.	0 valeur non entière.
`entrepôt absent : attendu outputs/entrepot.sqlite`	Chargement non lancé, ou lancé hors de `~/solen-bi`.	`python -m solen initial`, puis le contrôle.

Ce qu'on demande à Codex

Prompt 10, avec la commande exacte, le dossier, votre prévision et **toute** la sortie. Une ligne ÉCHEC à la fois, en commençant par la première. Correction dans `solen/` ; si Codex propose de modifier `controles/` ou `attendu_lot2.json`, refusez. Puis relancez **toutes** les étapes acquises (`initial`, `increment`, `rejeu`) et `pytest -q`.

03 / DIRIGER CODEX · RÉCAPITULATIF

À ce stade, je sais…

Je sais…	Je le prouve en…
…dire qui fait quoi : Codex propose, nous décidons, Git est notre geste	refusant une proposition qui touche `controles/` ou une source
…distinguer modèle, effort, vitesse, tokens, contexte et quota	choisissant l'effort avec la fiche tâche, et en lisant l'usage affiché sans rien promettre
…appliquer les huit habitudes	montrant une demande précise, un plan court, une note de journal avec une preuve réelle
…adapter un prompt et prévoir sa valeur	écrivant ma prévision avant d'envoyer, puis en la comparant à la sortie relancée
…lire un diff et situer une fonction, une exception, une classe	montrant où une ligne est rejetée et où les millimes sont calculés
…relire la PR de mon binôme	écrivant une revue GitHub avec une valeur que j'ai reproduite, distincte de l'avis de Codex
…faire le cycle Git et diagnostiquer une erreur	réparant un commit sur `main` ou un conflit sans `--force` ni `reset --hard`
…continuer sans Codex	faisant Git, revue, calcul à la main et note d'état pendant une limite d'usage
…lire une erreur Python ou un ÉCHEC	expliquant « attendu X, observé Y » et la question que je pose à Codex

Auto-contrôle (sans relire le livret)

1. Conversation longue, Codex oublie les millimes : effort, contexte ou quota ? 2. Que vérifiez-vous dans `git status` avant `git add` ? 3. Codex dit « aucun défaut établi » : pouvez-vous approuver ? 4. `UNIQUE constraint failed: fact_vente.ligne_id` après un deuxième `increment` : quelle règle est violée ? Comparez vos réponses avec votre binôme.

Prochaine étape : gardez le livret 06 (aide-mémoire) ouvert pendant les séances, et revenez ici à chaque nouvelle demande et à chaque PR.